Here is how a strong candidate would walk through designing a URL shortener in a system design interview, structured to drive the conversation from requirements to a concrete, scalable architecture.

1. Requirement Clarification & Scope Estimation

Before diving into the architecture, I'll clarify the core requirements and establish the scale we are designing for.

Functional Requirements

- **Shorten URL:** The system takes a long URL and returns a unique, shortened URL (e.g., bit.ly/AbC123).
- **Redirection:** When a user hits the short URL, the system redirects them to the original long URL with a 302 Found (or 301 Moved Permanently) HTTP status code.
- **Custom Aliases:** Users can optionally provide a custom short slug (e.g., bit.ly/my-promo).
- **Expiration:** URLs should have a default lifetime (e.g., 2 years) or an optional user-defined expiration date.

Non-Functional Requirements

- **High Availability:** The system must be highly available; if the redirection service goes down, links break globally.
- **Low Latency:** Redirection must happen instantly (sub-100ms response time).
- **Uniqueness:** Generated short links must be unique and unpredictable to prevent trivial enumeration attacks.

Capacity Estimation

Let's assume the scale of the system is similar to a major global shortener:

- **New URLs generated:** 100 million per month.
- **Read-to-Write Ratio:** Highly read-heavy. Let's assume a **100:1** read-to-write ratio.
  - **Writes:** $\approx 40 \text{ URLs/second}$ ($\frac{100,000,000}{30 \times 24 \times 3600}$)
  - **Reads (Redirections):** $40 \times 100 = 4,000 \text{ requests/second}$ (RPS).

Storage Estimation (for 5 years):

- Total URLs = $100\text{M/month} \times 12\text{ months} \times 5\text{ years} = 6 \text{ billion URLs}$.
- Assuming each record (Long URL, Short URL, Created At, Expires At) takes around **500 bytes**:
  - Total Storage = $6\text{ billion} \times 500\text{ bytes} \approx 3 \text{ Terabytes}$. This is a manageable dataset size, meaning storage volume isn't our primary bottleneck, but throughput and latency are.

2. API Design & Data Model

API Endpoints

We can expose a RESTful interface for the two primary operations:

1. **Create Short URL:**
   POST /api/v1/shorten
   - Request Body: {"long_url": "https://example.com/very/long/path", "custom_alias": "optional-slug", "expire_at": "2028-06-13T23:42:15Z"}
   - Response: {"short_url": "https://tiny.url/AbC123Z"}
2. **Redirection:**
   GET /{short_url_slug}
   - Response: 302 Found with HTTP Header Location: https://example.com/very/long/path

Interview Note: I chose 302 Found (temporary redirect) instead of 301 Moved Permanently. While 301 reduces server load because the browser caches the redirect, 302 ensures that every click hits our backend, allowing us to accurately track real-time analytics and metrics.

Data Model

Since the data is highly structured, mapping a unique short string to a long URL, and we don't require complex relational joins, we can leverage a NoSQL Key-Value store like **DynamoDB** or a wide-column store like **Cassandra** for seamless horizontal scaling.
Our core table schema looks like this:

Column Name	Type	Description
short_slug (PK)	String	The 7–8 character short key (e.g., AbC123Z)
original_url	String	The target destination URL
created_by	String	User ID (for rate limiting/analytics)
created_at	Timestamp	Generation time
expires_at	Timestamp	TTL marker for auto-deletion

3. High-Level Architecture & Core Logic

The backbone of this system relies on how we generate the unique short_slug. We need a mechanism that works reliably at scale without creating distributed race conditions.

The Shortening Algorithm (Base62 Encoding)

To keep the URL short, we can use **Base62 encoding** ([a-z, A-Z, 0-9]).

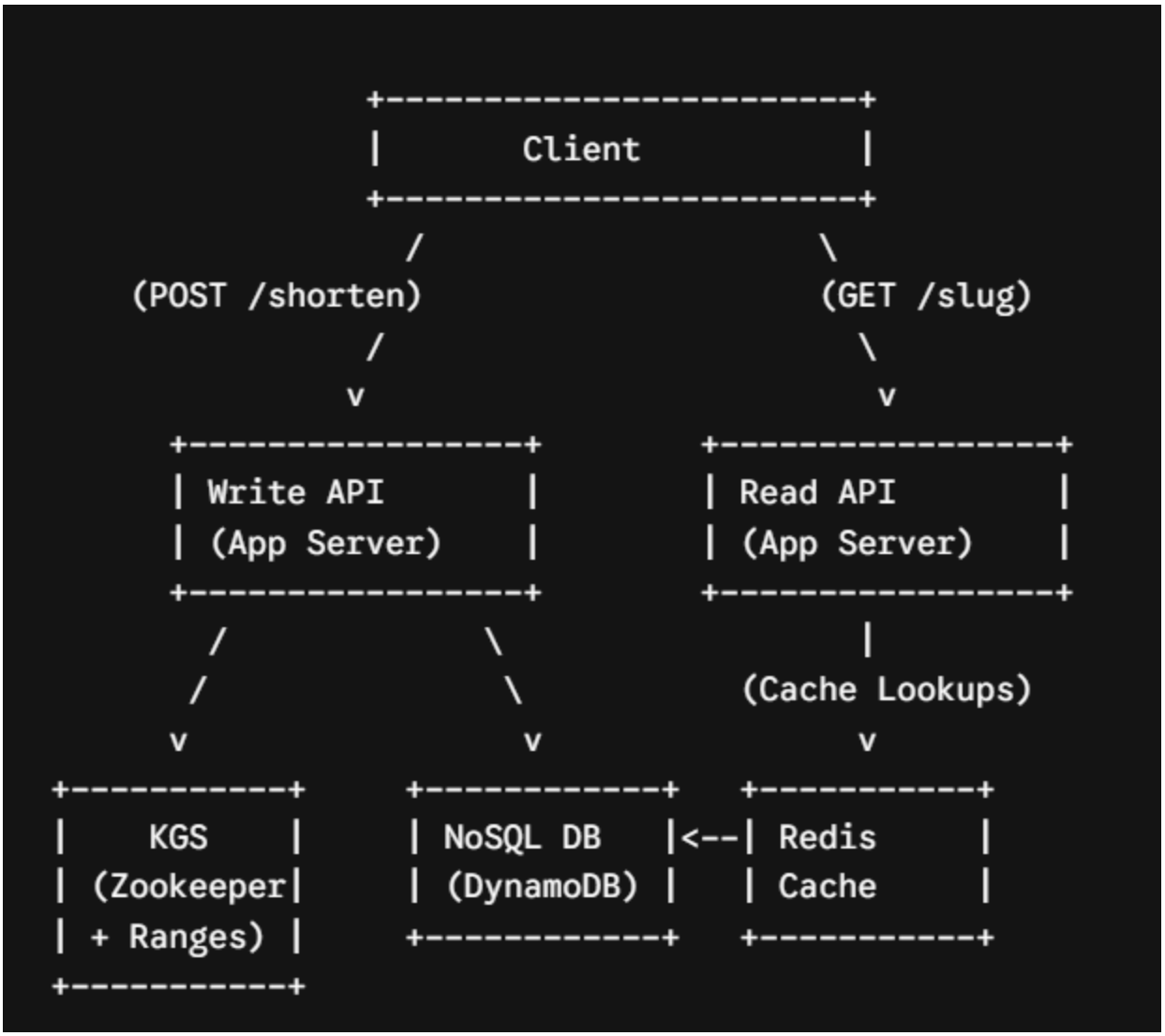
- A slug length of **7 characters** gives us $62^7 \approx 3.5 \text{ trillion}$ unique combinations, which easily satisfies our 6 billion URL requirement.

How do we assign a unique base62 string to a new write request?

- **Approach A: Hashing (MD5/SHA-256):** Hash the original URL and take the first 7 characters. *Problem:* Collision risk requires us to continually check the database, increasing write latency.
- **Approach B: Token/Unique ID Generator (Recommended):** Convert a unique auto-incrementing 64-bit integer into its Base62 equivalent. For example, ID 10,000,000 becomes fxMS.

To implement Approach B without creating a single point of failure (like a single MySQL auto-incrementing database), we will use a **Key Generation Service (KGS)**.

High-Level Components



1. **Load Balancer (Layer 7):** Distributes incoming read/write traffic across multiple stateless

web app servers.
2. **Key Generation Service (KGS):** Maintains a central coordinator (like **Apache ZooKeeper**) to allocate ranges of IDs (e.g., Server A gets IDs 1–1,000,000; Server B gets 1,000,001–2,000,000). The web servers can then generate unique IDs sequentially entirely in-memory. If a server crashes, its remaining chunk of IDs is lost, but because our ID space (3.5T) is so large, this loss is negligible.
3. **Database Layer:** A distributed NoSQL database storing the mappings.
4. **Caching Layer:** A Redis cluster placed in front of the Read API.

4. Deep Dive & Scaling the Design

Optimizing Read Latency (The Redirection Flow)

Because our system is heavily read-biased (4,000 RPS vs 40 WPS), we need to ensure reads rarely hit the persistent database.

- **Redis Caching:** We will cache the top **20%** of most frequently accessed links (applying the Pareto Principle).
- **Eviction Policy:** We will use a **Least Recently Used (LRU)** eviction policy.
- **Flow:** When a request for tiny.url/AbC123Z arrives, the read server queries Redis. On a cache hit, it issues a 302 redirect immediately. On a cache miss, it reads from the NoSQL database, populates Redis, and redirects.

Data Partitioning and Sharding

While 3TB fits on a single modern drive, we shard the database across multiple nodes to handle high concurrent read/write throughput.

- **Hashing Shard Key:** We will shard our NoSQL database using the short_slug as the partition key.
- Consistent hashing will ensure an even distribution of write and read requests across the database nodes, preventing hot spots during peak traffic.

Purging Expired Links

To keep storage optimized, we shouldn't continuously scan the database for expired entries.

- **Lazy Deletion:** When a user requests a link, we check the expires_at timestamp. If it has passed, we delete the record and return a 404 Not Found.
- **Background Sweeper:** A low-priority asynchronous cron job or native NoSQL TTL feature (like DynamoDB TTL) will run during off-peak hours to lazily sweep and delete expired rows from the database and cache.

Handling Custom Aliases

If a user requests a custom slug like /my-promo:

1. The Write API directly checks if my-promo already exists in the database.
2. If it is available, it skips the KGS step and inserts the custom slug mapping directly. If it exists, it returns an HTTP 409 Conflict error.

5. Security & Edge Cases

- **Rate Limiting:** To prevent malicious actors from spamming the write endpoints or enumerating redirects, a rate-limiting layer (using a Token Bucket algorithm via API Gateway) will throttle clients by IP address or API key.
- **URL Sanitization & Blocking:** To prevent our platform from being used for phishing or malware distribution, we can asynchronously pass newly submitted URLs through a malware detection pipeline (e.g., Google Safe Browsing API) before activating the link.

This design delivers a low-latency, highly resilient system that scales predictably as write or read volumes grow.